E2P: Exploração Inicial de Aplicações Web com um Modelo de Linguagem Orientado por Ações

Henrique Oliveira da Cunha Franco[1], **Humberto Torres Marques Neto**[1]

[1]Instituto de Informática – Pontifícia Universidade Católica de Minas Gerais (PUC Minas)

henrique.franco.1448652@sga.pucminas.br, humberto@pucminas.br

Abstract. *This work investigates how far a locally executed language model can conduct initial exploration of an unfamiliar web application and observe potential failures. E2P integrates project inspection, bounded browser actions, state observations, test generation, and evidence-based review. The study characterizes 21 educational applications and validates the current prototype through repeated explorations, browser checks, and runtime reproduction. The model receives project information and live observations without a supplied navigation script or defect description. Four browser-verified properties distinguish reference behavior from altered implementations, and two application exceptions are reproduced in clean sessions. Eight additional explorations cover form validation, media, empty collections, and a component-based task application. They expose two runtime failures and an unsupported expectation, distinguishing exploration, observation, and defect assessment. Hardware capacity, context allocation, and inference time define the operating conditions and motivate further evaluation on more capable configurations.*

Resumo. *Este trabalho investiga até que ponto um modelo de linguagem executado localmente consegue realizar uma exploração inicial de uma aplicação web desconhecida e observar possíveis falhas. O E2P integra inspeção do projeto, ações limitadas no navegador, observações de estado, geração de testes e revisão de evidências. O estudo caracteriza 21 aplicações didáticas e valida a instância atual por explorações repetidas, verificações de navegador e reprodução de diagnósticos. O modelo recebe informações do projeto e observações da interface, sem roteiro de navegação ou descrição de defeito fornecidos. Quatro propriedades verificadas no navegador distinguem o comportamento de referência de implementações alteradas, e duas exceções das aplicações são reproduzidas em sessões limpas. Oito explorações adicionais abrangem validação de formulário, mídia, coleção vazia e uma aplicação de tarefas baseada em componentes. Elas expõem duas falhas de execução e uma expectativa sem sustentação, distinguindo exploração, observação e avaliação do defeito. Memória, alocação de contexto e tempo de inferência delimitam a configuração avaliada e orientam sua expansão.*

1. Introdução

Testes ponta a ponta (*End-to-End*, E2E) exercitam jornadas de uso sobre um sistema integrado. Sua utilidade depende não apenas de executar interações, mas também de

selecionar comportamentos relevantes e verificar seus resultados. Para uma aplicação web sem uma rotina de garantia de qualidade (*Quality Assurance*, QA), preparar essa automação envolve entender o produto, configurar seu ambiente, explorar suas funções, especificar casos e interpretar falhas. Automatizar somente a escrita de scripts deixa parte importante desse trabalho em aberto.

Modelos de linguagem de grande porte (*Large Language Models*, LLMs) têm sido investigados em geração de testes, reprodução de defeitos e apoio à manutenção de software [Wang et al. 2024, Kang et al. 2023]. Entretanto, uma resposta textual plausível não comprova que controles existem, que uma jornada foi executada ou que uma expectativa é correta. Estudos de agentes web evidenciam dificuldades na conclusão de tarefas realistas [Zhou et al. 2024, Drouin et al. 2024]. Em QA, essas dificuldades se somam ao risco de transformar um comportamento normal em relato de defeito.

Neste trabalho, *orientado por ações* caracteriza o papel do modelo na orquestração: a partir da observação da aplicação, ele escolhe uma operação em um catálogo explícito, executada e verificada por ferramentas. Não se trata de uma nova arquitetura neural ou de um modelo treinado neste estudo. O protótipo Action E2E Prototype (E2P) combina documentação e trechos de código, estado estruturado da interface, percepção visual opcional, ações limitadas e revisão humana. Seu propósito é apoiar a preparação de testes e a análise de evidências em aplicações web. A avaliação utiliza aplicações didáticas de pequeno porte com funções identificáveis, permitindo examinar como as decisões do modelo se relacionam com o comportamento observado.

O problema de pesquisa é: *até que ponto um modelo de IA consegue realizar uma exploração inicial de uma aplicação web desconhecida e, nesse processo, observar possíveis falhas?* O objetivo geral é investigar essa capacidade por meio do E2P, que conecta decisões do modelo à execução real da aplicação. Os objetivos específicos são: (i) reconhecer funções a partir de documentação, código e interface; (ii) selecionar e executar ações relacionadas ao propósito do produto; (iii) registrar estados e evidências que permitam revisar a trajetória; e (iv) analisar hipóteses de falha e os mecanismos utilizados para verificá-las.

A contribuição é uma implementação local desse processo e sua avaliação por trajetórias e verificações executáveis. O estudo apresenta as aplicações e seus comportamentos, examina a exploração realizada e confere propriedades da instância atual do instrumento. Também discute como a capacidade computacional condiciona o contexto disponível, a frequência de decisões e a profundidade potencial da investigação.

O restante do artigo apresenta a fundamentação e os trabalhos relacionados na Seção 2, a metodologia na Seção 3, a implementação na Seção 4, os resultados na Seção 5, as ameaças à validade na Seção 6 e as conclusões na Seção 7.

2. Referencial Teórico e Trabalhos Relacionados

2.1. Da execução de interações à avaliação de comportamento

Um caso de teste associa condições iniciais, ações e resultados esperados. Um oráculo é o critério utilizado para decidir se o resultado observado é aceitável. No E2P, critérios em linguagem natural antecedem a automação e podem ser revisados. Essa separação é

necessária porque a validade sintática do script, a existência do seletor e a aprovação da execução não garantem correspondência com a intenção do caso.

A revisão de Wang et al. [Wang et al. 2024] organiza aplicações de LLMs em diferentes atividades de teste e destaca desafios de sua integração com outras técnicas. A distinção entre gerar entradas, produzir asserções e avaliar resultados orienta este estudo: uma melhoria em uma dessas atividades não é automaticamente transferível às demais. Em particular, medir somente quantos testes passam favorece testes permissivos, que podem permanecer verdes mesmo diante de um defeito.

Na avaliação de hipóteses, distinguem-se três afirmações: uma interação ocorreu; seu efeito foi reproduzido; esse efeito viola uma expectativa justificável. As duas primeiras são observacionais. A terceira requer evidência normativa, como um requisito, uma mensagem explícita da interface ou uma regra de domínio sustentada pelo projeto. O código auxilia na explicação causal, mas não constitui sozinho uma especificação infalível: ele também pode conter o defeito investigado. Essa distinção fundamenta a análise dos falsos positivos apresentada adiante.

2.2. Agentes de interface e testes baseados em uso

WebArena oferece aplicações e tarefas web para avaliar conclusão funcional, em vez de apenas a plausibilidade do texto produzido. No experimento publicado, o melhor agente GPT-4 obteve 14,41% de sucesso, frente a 78,24% dos participantes humanos [Zhou et al. 2024]. Esses números pertencem àquele protocolo e não são uma referência quantitativa direta para o E2P. WorkArena e BrowserGym enfatizam ações padronizadas, observações multimodais e tarefas de software corporativo [Drouin et al. 2024]. Ambos reforçam a necessidade de ambientes registráveis e medidas ligadas ao comportamento efetivamente realizado.

No domínio móvel, GPTDroid estrutura a exploração como interação iterativa entre modelo e aplicação, utilizando informações da interface, feedback e memória relacionada às funcionalidades [Liu et al. 2024]. O E2P adota o princípio de observar novamente após agir, mas restringe decisões a identificadores de ações catalogadas e concentra-se em aplicações web. A memória de feedback humano entre execuções, descrita neste artigo, é diferente da memória da trajetória usada para escolher a próxima ação.

Avgust transforma vídeos de uso em uma representação de estados para apoiar a geração de testes em aplicações móveis [Zhao et al. 2022]. A aproximação com o E2P está na valorização de jornadas observadas. A diferença é a origem dessas jornadas: no E2P, elas são produzidas pela exploração guiada pelo modelo, e não fornecidas previamente como vídeos de usuários. Isso reduz a necessidade de uma gravação inicial, mas transfere ao explorador a responsabilidade de alcançar comportamentos úteis.

2.3. Geração, feedback e verificação de requisitos

LIBRO gera testes de reprodução a partir de relatos de defeitos e utiliza pós-processamento para avaliar e ordenar candidatos [Kang et al. 2023]. Enquanto esse trabalho parte de um problema relatado, o E2P também propõe hipóteses a partir da interface explorada. Portanto, além da reprodução, precisa avaliar se a própria alegação de erro é válida.

Ferreira et al. [Ferreira et al. 2025] apresentam um estudo industrial com geração de cenários Gherkin a partir de histórias de usuário e conversão para testes Cypress usando HTML. Os cenários foram considerados úteis em 95% das avaliações; 60% dos testes puderam ser utilizados como gerados, enquanto os demais exigiram ajustes, nova geração ou descarte. Esse resultado apoia o uso de artefatos intermediários revisáveis. Diferentemente daquele estudo, o E2P precisa inferir oportunidades de teste de um projeto e de sua execução, sem pressupor um conjunto completo de histórias de usuário.

Panta combina análise estática de fluxo e cobertura dinâmica para orientar iterativamente a geração de testes unitários [Gu et al. 2026]. TestWeaver usa recortes de programa, testes relacionados e anotações de execução para fornecer contexto dirigido ao modelo [Le et al. 2026]. Essas abordagens motivam o uso de sinais verificáveis e de contexto selecionado. Contudo, a cobertura de objetivos de interface do E2P não é cobertura de linhas ou ramos e não deve ser comparada numericamente a essas medidas.

GUISpector verifica requisitos em linguagem natural sobre protótipos de interface com um agente multimodal e disponibiliza feedback para supervisão [Kolthoff et al. 2025]. O E2P compartilha o interesse na rastreabilidade entre requisito e trajetória, mas também investiga a descoberta dos próprios casos. Isso torna a procedência da expectativa um problema central: inferir um requisito exige cautela adicional em relação a verificar um requisito fornecido.

SpecOps decompõe o teste de agentes GUI em geração, preparação, execução e validação. A avaliação relatada encontrou 164 defeitos em cinco agentes, com F1 de 0,89 [Ahmed et al. 2026]. Seu objeto são agentes de IA em ambientes reais, enquanto o E2P avalia aplicações web. A principal relação metodológica é a separação dos estágios; nem os resultados nem a autonomia daquela arquitetura podem ser atribuídos ao protótipo aqui estudado.

2.4. Posicionamento do trabalho

O E2P se posiciona na investigação da exploração inicial de aplicações web por um modelo de IA orientado por ações. Diante de um projeto apresentado sem roteiro de teste, o modelo precisa reconhecer sua finalidade, identificar possibilidades de interação e escolher como obter informação sobre seu funcionamento. A questão central é quanto dessa atividade pode ser realizada de forma útil e que indícios de falha podem surgir durante o percurso.

Os trabalhos relacionados oferecem fundamentos para essa investigação: agentes de interface sustentam o ciclo de observar e agir; geração de cenários inspira os artefatos intermediários; e estudos de reprodução e avaliação orientam a conferência das hipóteses. Essas contribuições informam a arquitetura do E2P e os critérios de análise. O objeto empírico deste trabalho é a atuação do modelo dentro dessa implementação, conectada a uma aplicação executada e a evidências de suas próprias decisões.

3. Metodologia

3.1. Delineamento e questões de pesquisa

A pesquisa é aplicada e exploratória. O E2P é utilizado para investigar o comportamento de um modelo de linguagem diante de uma aplicação web apresentada sem roteiro de navegação ou casos de teste previamente fornecidos. O contato inicial ocorre pela inspeção

do projeto e pela observação da interface; a partir desse contexto, o modelo escolhe ações, acompanha suas consequências e propõe verificações e hipóteses.

As questões de pesquisa são: **QP1**, quais funcionalidades o modelo reconhece e consegue exercitar durante uma exploração inicial; **QP2**, quais evidências permitem identificar possíveis falhas da aplicação; **QP3**, como os mecanismos de validação do E2P preservam a relação entre ação, resultado e expectativa; e **QP4**, como as condições de execução e os recursos computacionais delimitam a profundidade alcançada. O foco está na instância atual do protótipo e nas atividades que ela torna executáveis.

A avaliação articula três fontes de evidência: caracterização das aplicações e inspeção de seus artefatos; novas explorações com configuração mantida fixa; e verificações de navegador sobre propriedades determinadas. A trajetória produzida pelo modelo é analisada separadamente da propriedade exigida por um teste de regressão. Essa organização permite observar a iniciativa exploratória e conferir os mecanismos que sustentam sua interpretação.

3.2. Aplicações utilizadas e comportamentos de interesse

Para observar decisões em interfaces com finalidades diferentes, foram utilizados os exemplos públicos de Vanilla Web Projects [Traversy 2026]. O conjunto reúne 21 aplicações: formulários, listas, buscas, jogos, reprodução de mídia e recursos de voz. A escolha permite relacionar cada ação a um propósito identificável e examinar tanto operações locais quanto comportamentos dependentes de tempo ou de serviços externos. O catálogo Dopa [da Cunha Franco 2026], desenvolvido pelo próprio autor deste TCC, foi utilizado como cenário inicial de integração do protótipo; os exemplos públicos oferecem os casos de estudo apresentados nesta avaliação.

A caracterização considera organização do código, entradas, estado e dependências. Cada exemplo possui um documento HTML de entrada e dois a quatro arquivos-fonte, com 118 a 372 linhas físicas não vazias de HTML, CSS e JavaScript, incluindo comentários. Dependências instaladas, builds, mídia e arquivos minificados são excluídos da contagem. Os exemplos podem ser servidos estaticamente e não possuem backend próprio em suas pastas. Cinco utilizam chamadas `fetch`, quatro utilizam armazenamento no navegador e dois empregam APIs de fala.

Essas características descrevem aplicações de pequeno porte e escopo funcional delimitado. O porte é sustentado pela estrutura concentrada no navegador e pela quantidade restrita de responsabilidades, além do volume de código. A exigência de teste, por sua vez, depende do comportamento: uma interface de poucos arquivos pode envolver cálculo, estado acumulado, áudio ou temporização. Por isso, a análise apresenta as funções exercitadas e os resultados observáveis, sem utilizar linhas de código como uma escala de dificuldade.

A Tabela 1 apresenta as aplicações por finalidade predominante. As famílias organizam a leitura dos casos; capacidades como persistência e dependência de rede podem aparecer em mais de uma família. O inventário complementar associa cada aplicação a seus arquivos, medidas, comportamentos de referência e artefatos de execução.

Tabela 1. Aplicações utilizadas e comportamentos de interesse para exploração.

Família (n)	Aplicações e finalidade	Exigência de observação
Dados e formulários (5)	DOM Array Methods: listas; Expense Tracker: lançamentos; Form Validator: campos; Memory Cards: cartões; Movie Seat Booking: assentos.	Entrada, validação, alteração de valores e estado preservado.
Busca e seleção (5)	Exchange Rate: moedas; Infinite Scroll Blog: publicações; Lyrics Search: letras; Meal Finder: receitas; Product Filtering: produtos.	Consulta, resultados, seleção de detalhe e resposta de serviços.
Mídia e voz (4)	Custom Video Player: vídeo; Music Player: áudio; Speak Number Guess: reconhecimento; Speech Text Reader: síntese.	Controles, identidade do conteúdo e sinais de reprodução ou fala.
Jogos e temporização (5)	Breakout: colisões; Hangman: palpites; New Year Countdown: calendário; Relaxer App: respiração; Typing Game: digitação.	Teclado, sequências, pontuação e mudanças temporais.
Interação e ordenação (2)	Modal Menu Slider: painéis; Sortable List: lista reordenável.	Abertura, fechamento e alteração da ordem.

3.3. Exploração e coleta de evidências

Na exploração inicial, o modelo recebe documentação e trechos selecionados de código, estado estruturado da interface e, quando habilitadas, imagens da página. Não recebe uma sequência pronta de passos nem uma descrição de defeito a procurar. O E2P disponibiliza controles observados em um catálogo de ações. O modelo escolhe uma operação e seus parâmetros; depois da execução, uma nova observação registra a mudança e orienta a próxima decisão. A Figura 1 apresenta o processo.

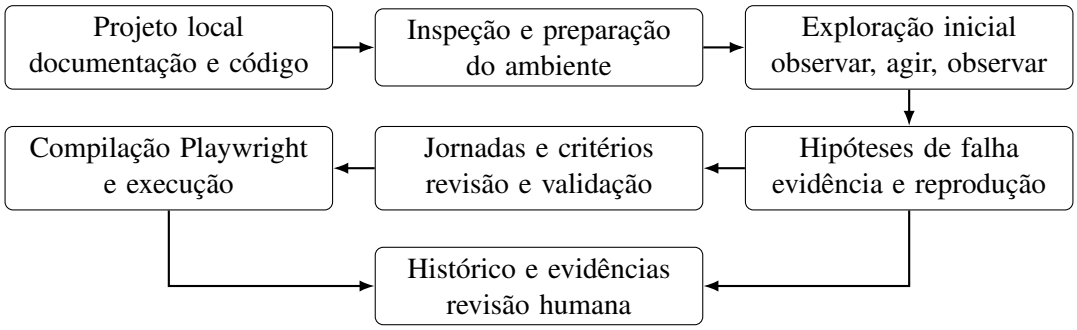


Figura 1. Exploração inicial e transformação das observações em artefatos revisáveis.

A validação atual inclui duas explorações por aplicação em Expense Tracker, Movie Seat Booking e Modal Menu Slider, alternando os alvos entre repetições. Os casos exercitam entrada tipada e cálculo, seleção com persistência e ciclo de painéis, respectivamente. O perfil de acesso é convidado, com dados fictícios e contexto de navegador isolado em cada tentativa. A implementação dos serviços é identificada por hashes e mantida fixa durante a coleta. Nenhuma jornada é fornecida ao modelo.

Uma rodada complementar utiliza Form Validator, Music Player, Memory Cards e TodoMVC React [TasteJS 2026], com duas repetições intercaladas e o mesmo perfil

de modelo. Os três primeiros ampliam os comportamentos examinados; o último acrescenta outra origem, componentes React, gerenciamento de estado e filtros por rotas, sem backend próprio. Continua sendo uma aplicação pequena, mas exige acompanhar uma coleção entre diferentes apresentações. Os fontes dos alvos e dos serviços permanecem fixos e identificados por hashes durante a coleta. O modelo não recebe gabarito, trajetória ou descrição de defeito.

O procedimento preserva ações, parâmetros, observações, estados e diagnósticos. Na análise, *exercitar* uma função requer executar sua operação e observar o efeito; *verificar* exige ainda confrontar o efeito com uma expectativa fundamentada. Selecionar um assento, conferir o total e testar restauração são atividades diferentes. A comparação de casos considera essas unidades, sem transformar contagens de passos em cobertura funcional.

3.4. Ambiente de inferência e recursos disponíveis

O modelo local de 27 bilhões de parâmetros utiliza os pesos `Qwen3.8-27B-UD-IQ1_M.gguf`, quantização IQ1_M e projetor visual BF16, com alias `qwen3.8-vl-27b-iq1m-64k`. O servidor llama.cpp b10819 opera com Vulkan, um slot, contexto configurado de 65.536 tokens, cache KV Q8 e Flash Attention [ggml-org 2026]. A máquina utiliza Windows, Ryzen 7 7800X3D, Radeon RX 9070 XT com 16 GB de memória gráfica e 32 GB de RAM. A exploração e os testes são executados por Playwright com Chromium.

A configuração procura acomodar inferência e automação na mesma máquina. A memória gráfica é compartilhada pelos pesos do modelo, cache de contexto e processamento visual; a RAM também atende o servidor, os processos da aplicação e o navegador. A quantização reduz o espaço ocupado pelos pesos e viabiliza executar esse perfil localmente. A janela configurada estabelece a capacidade máxima de contexto por chamada; o volume efetivamente utilizado depende da etapa e da seleção de evidências.

O orçamento da exploração é de até 20 ações e 180 segundos, com decisões e observações sucessivas. Assim, a profundidade depende tanto da qualidade das escolhas quanto do tempo de inferência, da resposta da aplicação e do custo de organizar o contexto. Selecionar evidências relevantes é parte do funcionamento do instrumento: código, histórico e imagens concorrem por espaço e processamento. O relógio da página é isolado durante decisões do modelo para reduzir a interferência da latência em interfaces temporizadas.

3.5. Validação e critérios de interpretação

A exploração é examinada por ações concluídas, funções reconhecidas, mudanças de estado e diagnósticos observados. A avaliação de um caso executável confronta a intenção do critério com sua asserção: entidade, valor esperado, comparação e momento de verificação. Um teste de persistência, por exemplo, precisa criar estado, recarregar e conferir o mesmo estado restaurado. A existência de um campo ou de um cabeçalho serve como evidência de interface, mas não substitui uma propriedade de domínio.

A implementação atual é submetida à suíte interna e a verificações de navegador com versões de referência e cópias modificadas dos alvos. São avaliadas mudança de faixa, atualização de saldo e armazenamento, além da preservação de um valor esperado

em formulário de controle com regra explícita de normalização. Em cada caso, a mesma suíte é executada no original e na cópia alterada; o efeito da alteração é conferido separadamente no navegador. Esses ensaios verificam a resposta das asserções, enquanto as explorações examinam as decisões do modelo.

Também são repetidas, em duas sessões limpas por aplicação, ações associadas a exceções no player de áudio e nos cartões. Para hipóteses de falha, a interpretação reúne ação executada, efeito observado, expectativa sustentada pelo projeto e resultado da reprodução. A análise dos candidatos é assistida por IA e inspeção dos artefatos. Uma revisão humana independente pode complementar essa classificação; o estudo apresenta contagens e evidências dos casos, sem estimar precisão ou revocação sobre um conjunto exaustivo de defeitos.

Os arquivos de acompanhamento registram configuração, identificação dos serviços, resultados e referências às evidências. O comando `evaluate:corpus` reproduz o inventário estrutural; os scripts de validação reproduzem as verificações da instância atual. Essa documentação permite examinar cada conclusão no nível de aplicação, ação e propriedade verificada.

4. Descrição da Implementação

4.1. Serviços e preparação do ambiente

O E2P utiliza Node.js e Express, interface HTML/CSS/JavaScript, Cheerio para inspeção estática e Playwright para observação e execução [Microsoft 2026]. Serviços separados tratam inspeção, ambiente, provedor, exploração, planejamento, geração, hipóteses e histórico. A inspeção seleciona documentação e trechos do repositório; o modelo contribui para sua interpretação, enquanto a execução depende de comandos e controles admitidos pelo instrumento. Essa separação torna possível atribuir uma falha ao estágio em que ela ocorreu.

O ambiente pode ser servido estaticamente, iniciado por comando, conectado a uma URL existente ou submetido apenas à análise. O comando e a URL inferidos são revisáveis. Cada execução mantém artefatos em diretório próprio, sem inserir testes no código do alvo. Instalar dependências e iniciar a aplicação ainda pode produzir efeitos no ambiente; a separação de diretórios não constitui isolamento completo.

O adaptador atende diferentes provedores; o lote principal usa llama.cpp diretamente. No modo convidado, capturas podem complementar o DOM quando o provedor suporta visão. Capturar uma imagem para inspeção e enviá-la ao modelo são operações distintas. O modo autenticado é uma prova de conceito somente para leitura, com referências a segredos por variáveis de ambiente, sessões isoladas e quarentena de artefatos; vídeos e rastros ficam desabilitados nesse modo.

4.2. Exploração limitada e representação de estado

A cada observação, o explorador constrói um catálogo de controles com identificadores, nomes acessíveis e operações admitidas. O modelo escolhe nesse catálogo; não fornece um seletor arbitrário para execução irrestrita. Respostas JSON e decisões inválidas recebem correção limitada. A política AI-first interrompe a progressão quando falta participação válida do modelo, preservando a evidência parcial.

Objetivos QA derivados da inspeção contextualizam entrada, seleção, navegação, validação, temporização, persistência e cálculo. O perfil dos registros limita a exploração a até 20 ações e 180 segundos; esperas explícitas são limitadas a 12 segundos. Uma recarga depende de indícios de persistência e mudança anterior. Essas oportunidades orientam a descoberta, mas sua ocorrência não é uma asserção de correção.

O relógio da página convidada é pausado durante a inferência e retomado antes da ação. Isso reduz mudanças silenciosas de estado enquanto o modelo decide; não congela serviços externos nem retira a inferência do orçamento total. A política prioriza entradas fictícias ordinárias, com correção limitada após validação e respeito aos limites declarados pelos controles. Encerrar por esgotamento do catálogo significa esgotar ações permitidas, não compreender todo o produto.

4.3. Planejamento, compilação e execução

Fluxos e critérios devem remeter a controles e transições registrados. O planejamento recebe resumos de ações e estados com identificadores, reduzindo repetição de catálogos extensos no contexto. A revisão permite selecionar e editar critérios. Em lote, são admitidos fluxos fundamentados de confiança média/alta ou, no modo convidado, um teste explicitamente limitado de transição observada, desde que haja ação concluída, mudança, controle estável e resultado citado.

No caminho convidado, o gerador compila a jornada executada em Playwright, no modo `model-journey-compiled`. O modelo escolhe a trajetória e participa da autoria semântica; o compilador determina a estrutura executável e as verificações disponíveis. Portanto, a qualidade do teste depende de duas decisões: o que o explorador alcança e o que a tradução conserva. Um seletor fundamentado pode apontar corretamente para um campo e ainda verificar a propriedade errada.

Na instância avaliada, um contrato registra a origem da expectativa, a ação, a comparação, a evidência e a identificação da asserção. Critérios editados são revalidados; os não suportados permanecem não verificados. O contrato torna a tradução inspecionável. O executor distingue término operacional, falha de asserção e erro de infraestrutura. Relatório ausente não equivale a sucesso, e cobertura com denominador vazio é não mensurável.

4.4. Hipóteses, reprodução e memória

Um papel autor propõe hipóteses a partir da trajetória; um papel crítico examina ação, expectativa e efeito alegado. Regras determinísticas filtram candidatos e podem registrar exceções do navegador. Dois papéis não implicam modelos distintos nem julgamentos independentes. Hipóteses elegíveis passam por reprodução em contexto limpo, repetindo ações conhecidas e comparando o efeito; isso não estabelece, sozinho, que a expectativa era válida.

O coletor registra diálogos nativos vinculados à ação. Ela preenche uma lacuna importante: ausência de mudança no DOM não demonstra ausência de feedback. Entradas numéricas fictícias exigem um perfil descartável local explícito. Seus valores respeitam os limites e o incremento declarados no HTML; credenciais continuam sujeitas à política de acesso.

O histórico preserva etapas, erros, testes, capturas e hipóteses em `run-log.json` e artefatos associados. Revisões humanas são eventos com justificativa; decisões posteriores não apagam as anteriores. Até 20 revisões recentes por caminho de projeto podem contextualizar autor e crítico. Esse recurso recupera contexto, não treina o modelo, e não foi demonstrado que reduza falsos positivos. Feedback anterior não substitui evidência da execução corrente.

4.5. Instrumentação para avaliação

O módulo de caracterização associa alvos a logs e arquivos, produzindo resultados por família, medidas estruturais, hashes e referências às asserções. A associação ao lote é validada antes da agregação. Comportamentos de referência anotados pelo avaliador permanecem separados do resultado automático e da confirmação de hipóteses. O suplemento publica protocolos, trajetórias e verificações selecionadas; scripts reproduzem os ensaios sem exigir um histórico privado. Esse material permite conferir as conclusões no nível da aplicação, da ação e da propriedade avaliada.

5. Resultados e Discussão

5.1. Exploração inicial na instância atual

As novas explorações examinam a capacidade do modelo de iniciar o contato com a aplicação, reconhecer controles e produzir uma sequência de ações relacionada à finalidade do produto. A Tabela 2 apresenta as duas repetições de cada aplicação, sob a mesma configuração dos serviços. A contagem considera somente ações concluídas; a descrição da trajetória permite verificar o que esses números representam.

Tabela 2. Explorações da instância atual: ações concluídas e estados por repetição.

	Ações		Estados	
Aplicação	R1	R2	R1	R2
Expense Tracker	5	5	4	4
Movie Seat Booking	17	17	18	18
Modal Menu Slider	8	8	4	4

As seis explorações registraram 60 ações concluídas e 52 estados somados entre as tentativas. Os identificadores distinguem observações dentro de cada execução; a soma não representa funcionalidades independentes. A conferência dos hashes confirmou que os serviços permaneceram inalterados durante a coleta.

Nas duas explorações do registro financeiro, o modelo preencheu a descrição *Lunch*, informou 15,00, confirmou o formulário e recarregou a página. O histórico manteve o lançamento e o saldo observado foi 15,00. A trajetória alcançou, assim, criação e restauração de estado de domínio. O catálogo informou os limites numéricos e o incremento aceito pelo campo, permitindo que o modelo escolhesse um valor sintético compatível com a validação nativa.

No seletor de assentos, a oportunidade exploratória é combinar escolha de filme, seleção de lugares e estado preservado. No exemplo de menu e diálogo, o interesse está

no ciclo de abertura, interação e fechamento dos painéis. Esses casos mostram por que a finalidade da aplicação orienta a leitura dos resultados: cinco ações sobre campos têm um significado diferente de cinco ações de navegação entre estados.

As trajetórias são propostas pelo modelo a partir dos sinais encontrados. Documentação, código e interface fornecem contexto sobre o produto, enquanto a execução confirma a existência dos controles e o efeito das decisões. Essa combinação caracteriza o trabalho exploratório inicial investigado: reconhecer oportunidades, experimentar operações e registrar observações que possam fundamentar verificações posteriores.

5.2. Exploração em contextos complementares

A Tabela 3 apresenta as oito explorações adicionais, com 38 ações concluídas e 26 estados somados entre execuções. Esses totais descrevem atividade, não funções independentes. As duas repetições permitem examinar trajetórias na configuração adotada; sua proximidade também deve ser lida à luz da temperatura zero utilizada nas decisões estruturadas.

Tabela 3. Rodada complementar: ações e estados por repetição (R1/R2).

Aplicação	Ações	Estados	Alcance observado
Form Validator	2/2	3/3	Preenchimento de Username e submissão por Enter; mensagens dos demais campos.
Music Player	3/3	2/2	Navegação de faixa, controle de progresso e reprodução; exceção capturada.
Memory Cards	9/9	3/3	Entradas, limpeza, recarga e painéis; navegação sem cartão produz exceção.
TodoMVC React	5/5	5/5	Criação de tarefa e alternância entre All, Completed e Active.

No TodoMVC, a tarefa *Buy groceries* aparece após Enter, deixa de aparecer no filtro Completed e retorna em Active. A sequência relaciona entrada, criação e seleção de subconjunto. Ela não exercita edição, conclusão nem restauração após recarga. Nos cartões, preencher pergunta e resposta não comprova a criação: o percurso não concluiu essa operação. Esses contrastes mostram que diversidade de ações, funções alcançadas e correção verificada precisam ser analisadas separadamente.

No formulário, o explorador preenche somente Username, pois a política restringe campos de e-mail e senha. Uma repetição gerou a hipótese de que Enter deveria validar apenas esse campo; a outra não a reteve. Uma conferência no navegador mostrou que Enter e o botão Submit disparam a mesma submissão e as mesmas três mensagens. O código valida o formulário completo nesse evento. Assim, a observação é reproduzível, mas a expectativa de validação individual não é sustentada: o caso requer descarte fundamentado, e não confirmação por similaridade de estado.

5.3. Validação das propriedades observadas

A suíte interna da versão atual contém 152 testes aprovados, abrangendo contratos de decisão, observação, evidência, execução e apresentação dos resultados. Foram acrescentados casos que confrontam valor esperado e observado, identidade do campo, correspondência entre ação e estado, ambiguidade do critério e coerência do crítico. O teste de

navegador também confere que a validação nativa e seus valores são observados no perfil descartável.

A Tabela 4 sintetiza quatro propriedades verificadas com a mesma asserção em versões de referência e cópias alteradas. O teste aceita o comportamento esperado e falha por asserção quando a propriedade é violada. No registro financeiro, os testes de saldo e persistência são independentes: desabilitar uma dessas propriedades não faz o outro teste falhar.

Tabela 4. Validação da versão atual em propriedades verificáveis no navegador.

Propriedade	Diferença introduzida na cópia	Referência	Alterada
Identidade da faixa	Desativar a ação de faixa anterior	Passa	Falha
Saldo do lançamento	Desativar a atualização do saldo	Passa	Falha
Lançamento após recarga	Desativar o armazenamento local	Passa	Falha
Valor aprovado no critério	Produzir 42 quando o requisito exige 99	Passa	Falha

O formulário de controle verifica uma propriedade central do contrato. Mesmo quando a trajetória registrada mostra 42, o critério aprovado exige 99; o teste gerado conserva 99 como resultado esperado. A implementação de referência aplica essa regra e passa, enquanto a cópia que mantém 42 falha. Esse procedimento confirma que a observação do alvo não substitui automaticamente a expectativa utilizada para avaliá-lo.

A identidade do campo e do estado também é validada. Um campo diferente com o mesmo valor não fundamenta a asserção do controle escolhido, e um estado sem correspondência com a ação não é utilizado como seu resultado. Critérios com alternativas ambíguas permanecem disponíveis para revisão. A contribuição desses mecanismos é tornar explícita a relação entre ação, expectativa e comparação executada.

Para o papel crítico, frases afirmativas que sustentam uma hipótese são aceitas como tal, enquanto justificativas negativas incompatíveis com os campos estruturados são detectadas. Candidatos que repetem o mesmo resultado como esperado e observado são rejeitados por não descreverem uma violação. Essa regra foi reaplicada às hipóteses dos painéis, preservando as trajetórias originais para conferência. A verificação complementa as exigências de ação observada e fonte para a expectativa.

5.4. Observação e reprodução de falhas

Na rodada complementar, ações escolhidas pelo modelo expuseram as duas exceções em ambas as repetições. No Music Player, a atualização de tempo tenta escrever em elementos ausentes, produzindo `Cannot set properties of null (setting 'innerHTML')`; o código relaciona o efeito às referências `currTime` e `durTime`. No Memory Cards, navegar sem cartão produz `Cannot set properties of undefined (setting 'className')`, em acesso a `cardsEl[currentActiveCard]`.

A atribuição dos papéis é importante: o modelo seleciona as ações; o coletor do E2P identifica a exceção; a reprodução e a inspeção relacionam diagnóstico, trajetória e causa. O resultado demonstra alcance de estados defeituosos pelo sistema combinado,

sem atribuir ao modelo um diagnóstico que veio da instrumentação. Não se estima revocação sobre todos os defeitos dessas aplicações.

Uma verificação adicional reproduziu as quatro trajetórias com diagnóstico em sessões limpas. O reprodutor passou a tratar recarga e espera como operações do navegador, em vez de procurar controles DOM correspondentes. Isso permitiu completar as duas trajetórias dos cartões, inicialmente bloqueadas nesse ponto. Os registros originais e essa conferência permanecem separados no suplemento: repetir uma trajetória gravada verifica reprodução, não acrescenta uma nova descoberta.

Outras modalidades exigem canais de observação próprios. Uma mensagem de validação em diálogo nativo pode não alterar o texto do DOM; por isso o E2P registra o tipo, a mensagem e a ação associada ao diálogo. Uma falha de serviço externo exige verificar a resposta recebida e seu tratamento. Da mesma forma, texto preenchido em uma interface de fala é evidência de entrada, enquanto síntese efetiva depende da observação da API ou da saída audível. Essas distinções orientam a análise da falha e a escolha de uma asserção apropriada.

5.5. Profundidade exploratória e capacidade computacional

A inferência e a automação compartilham memória e tempo. Pesos, cache KV, projetor e evidências visuais competem pela VRAM; o contexto reservado de 65.536 tokens é capacidade, não medida de consumo efetivo. Mais memória pode viabilizar menor quantização ou mais evidência por decisão; maior velocidade pode permitir mais decisões no mesmo intervalo. Esses efeitos precisam ser medidos separadamente.

A transferência para uma RTX 3060 com i5-10400F e 16 GB de RAM foi preparada com perfis menores, mas ainda não constitui resultado experimental. Uma comparação informativa mantém modelo, quantização, alvos e orçamento, variando primeiro o contexto e depois a máquina. Quando a memória exige trocar o modelo, o resultado descreve configurações completas. Devem ser registrados tokens utilizados, memória, latência e funções alcançadas, além de ações e estados.

5.6. Respostas às questões de pesquisa

Quanto à **QP1**, as trajetórias mostram a iniciativa do modelo na identificação e no exercício de operações a partir da documentação e da interface. As repetições permitem examinar a estabilidade dessa iniciativa na mesma configuração. O resultado deve ser lido no nível das funções alcançadas, pois quantidade de passos e quantidade de estados representam aspectos diferentes da exploração.

Para a **QP2**, as explorações adicionais alcançaram estados com exceções no player e nos cartões; a instrumentação registrou os erros e as quatro trajetórias puderam ser reproduzidas. O formulário mostrou a situação complementar: uma observação repetível associada a uma expectativa sem sustentação. Portanto, a investigação de falhas depende tanto de alcançar o estado quanto de justificar a violação.

A **QP3** é atendida pelos contratos e ensaios que preservam valores aprovados, identidade da entidade e correspondência com a ação. As quatro propriedades verificadas no navegador demonstram discriminação entre o comportamento de referência e sua alteração. Para a **QP4**, a execução local caracteriza o equilíbrio entre memória, contexto e

tempo: o protótipo realiza a exploração nessa configuração, e sua expansão requer medir como recursos adicionais afetam as decisões e a abrangência obtida.

6. Ameaças à Validade

A interpretação considera o escopo dos exemplos, a configuração local e as evidências produzidas. As duas repetições por aplicação permitem examinar estabilidade nessa configuração; maior variedade de aplicações e repetições pode ampliar a caracterização do comportamento exploratório. Ações e estados são acompanhados de sua finalidade para evitar que uma contagem de interações seja interpretada como cobertura funcional completa.

O controle de estados iniciais e a comparação entre referência e alteração sustentam os ensaios de asserção. Nas aplicações com rede, temporização ou dados variáveis, essas condições devem ser registradas para interpretar diferenças entre execuções. A reprodução de uma exceção confirma seu efeito no ambiente observado; a classificação de outras hipóteses depende também de uma expectativa justificável e pode ser fortalecida por revisão humana independente.

Os resultados descrevem aplicações didáticas e uma aplicação React de tarefas, em modo convidado e Chromium, com um perfil local de modelo. A política de ações também condiciona o alcance: o formulário não é preenchido integralmente, e edição, conclusão e persistência da tarefa não foram exercitadas. A análise em sistemas com mais módulos, papéis de acesso e persistência de servidor constitui uma extensão do recorte. Recursos computacionais e contexto efetivamente utilizado também precisam ser considerados ao transferir o procedimento para outra máquina.

7. Conclusões e Trabalhos Futuros

O E2P permite investigar a exploração inicial de uma aplicação web por um modelo de linguagem, conectando reconhecimento de funções, escolha de ações, observação de estados e análise de possíveis falhas. As trajetórias registradas mostram como o modelo utiliza sinais do projeto e da interface para experimentar operações sem um roteiro previamente fornecido. A execução real e os artefatos tornam essas decisões examináveis.

A validação reúne 14 explorações em sete aplicações, quatro propriedades de asserção e reprodução de diagnósticos. A rodada complementar mostrou criação e filtragem de tarefas, estados com exceções e uma hipótese cuja expectativa não era sustentada. Em relação aos agentes web da literatura, o resultado enfatiza a iniciativa sem tarefa previamente especificada; em relação à geração de testes, explicita a origem e a conferência do oráculo. Os protocolos distintos não permitem comparar taxas de sucesso numericamente. A contribuição é conectar essas etapas em um instrumento local e rastreável.

Como continuidade, propõe-se utilizar o **JEV**, da TypeSafe, como componente de decisão e avaliação estruturada [TypeSafe AI 2026]. O modelo recebe estado e perguntas tipadas, retornando respostas e distribuições de probabilidade. No E2P, pode ser investigado para selecionar entre ações catalogadas, avaliar se uma evidência sustenta determinada hipótese e encaminhar casos incertos à revisão humana. A avaliação dessa integração deve comparar concordância com julgamentos humanos, custo, latência e comportamento em casos ambíguos, com limiares definidos para cada decisão.

Outra direção é avaliar máquinas com maior memória gráfica, RAM e capacidade de inferência. Essa ampliação pode permitir modelos menos quantizados, maior contexto útil e mais alternativas de exploração em tempo viável. O procedimento deve comparar separadamente recursos, configuração do modelo e orçamento de ações, medindo funções alcançadas, estabilidade e evidências de falha. A expansão para aplicações com jornadas integradas e diferentes perfis de acesso complementa essa análise e amplia o alcance do processo exploratório.

Declaração de uso de inteligência artificial

A ferramenta Codex, da OpenAI, foi utilizada para apoio à revisão e atualização deste artigo, incluindo organização textual, tradução do resumo, conferência bibliográfica, confronto com código e artefatos experimentais e formatação LaTeX. A contribuição abrangeu todas as seções desta versão. Os registros de implementação e avaliação também incluem operações assistidas por IA; não são apresentados como estudo independente com participantes humanos. A responsabilidade pela exatidão, originalidade, interpretação dos resultados e aprovação da versão submetida permanece com o autor e seu orientador.

Referências

Ahmed, S. Y., Feng, S., Bae, C., Barrus, C., and Zhang, X. (2026). SpecOps: A fully automated AI agent testing framework in real-world GUI environments. In *Proceedings of the 48th International Conference on Software Engineering (ICSE)*. https://doi.org/10.1145/3744916.3787778.

da Cunha Franco, H. O. (2026). *Dopa*. Repositório de software. https://github.com/pieceofhell/dopa. Cópia local utilizada na avaliação de 8 set. 2026.

Drouin, A., Gasse, M., Caccia, M., Laradji, I. H., Del Verme, M., Marty, T., Boisvert, L., Thakkar, M., Cappart, Q., Vazquez, D., Chapados, N., and Lacoste, A. (2024). WorkArena: How capable are web agents at solving common knowledge work tasks? arXiv:2403.07718, versão 5. https://arxiv.org/abs/2403.07718.

Ferreira, M., Viegas, L., Faria, J. P., and Lima, B. (2025). Acceptance test generation with large language models: An industrial case study. In *IEEE/ACM International Conference on Automation of Software Test (AST)*. https://doi.org/10.1109/AST66626.2025.00007.

ggml-org (2026). *llama.cpp*. Software. https://github.com/ggml-org/llama.cpp. Acesso em: 13 set. 2026.

Gu, S., Nashid, N., and Mesbah, A. (2026). LLM test generation via iterative hybrid program analysis. In *Proceedings of the 48th International Conference on Software Engineering (ICSE)*. https://doi.org/10.1145/3744916.3764553.

Kang, S., Yoon, J., and Yoo, S. (2023). Large language models are few-shot testers: Exploring LLM-based general bug reproduction. In *Proceedings of the 45th International Conference on Software Engineering (ICSE)*. https://arxiv.org/abs/2209.11515.

Kolthoff, K., Kretzer, F., Ponzetto, S. P., Maedche, A., and Bartelt, C. (2025). GUISpector: An MLLM agent framework for automated verification of natural language

requirements in GUI prototypes. arXiv:2510.04791. https://arxiv.org/abs/2510.04791.

Le, C. C., Van, C. D., Vu, T. D., Vu, T. M. P., Phan, H. N., Phan, H. N., and Nguyen, T. N. (2026). TestWeaver: Execution-aware, feedback-driven regression testing generation with large language models. In *Proceedings of the 48th International Conference on Software Engineering (ICSE)*. https://arxiv.org/abs/2508.01255.

Liu, Z., Chen, C., Wang, J., Chen, M., Wu, B., Che, X., Wang, D., and Wang, Q. (2024). Make LLM a testing expert: Bringing human-like interaction to mobile GUI testing via functionality-aware decisions. In *Proceedings of the 46th International Conference on Software Engineering (ICSE)*. https://arxiv.org/abs/2310.15780.

Microsoft (2026). *Playwright: Assertions*. Documentação. https://playwright.dev/docs/test-assertions. Acesso em: 13 set. 2026.

TasteJS (2026). *TodoMVC: React example*. Repositório de software. https://github.com/tastejs/todomvc/tree/master/examples/react. Cópia local identificada no suplemento; acesso em: 20 out. 2026.

Traversy, B. (2026). *Vanilla Web Projects*. Repositório de exemplos. https://github.com/bradtraversy/vanillawebprojects. Cópia local utilizada nas avaliações de set. 2026.

TypeSafe AI (2026). *Jev: Introduction*. Documentação oficial. https://docs.typesafe.ai/introduction. Acesso em: 18 out. 2026.

Wang, J., Huang, Y., Chen, C., Liu, Z., Wang, S., and Wang, Q. (2024). Software testing with large language models: Survey, landscape, and vision. *IEEE Transactions on Software Engineering*. Versão dos autores: https://arxiv.org/abs/2307.07221.

Zhao, Y., Talebipour, S., Baral, K., Park, H., Yee, L., Khan, S. A., Brun, Y., Medvidović, N., and Moran, K. (2022). Avgust: Automating usage-based test generation from videos of app executions. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. https://arxiv.org/abs/2209.02577.

Zhou, S., Xu, F. F., Zhu, H., Zhou, X., Lo, R., Sridhar, A., Cheng, X., Ou, T., Bisk, Y., Fried, D., Alon, U., and Neubig, G. (2024). WebArena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/2307.13854.